

```
1 coins = [1, 5, 10, 25]
2 amount = 63
3
4 coins.sort(reverse=True)
5
6 result = []
7
8 for coin in coins:
9     while amount >= coin:
10         amount -= coin
11         result.append(coin)
12
13 print("Coins:", *result)
14 print("Minimum coins =", len(result))
```

Coins: 25 25 10 1 1 1
Minimum coins = 6

...Program finished with exit code 0
Press ENTER to exit console.

```
1 weights = [10, 20, 30]
2 profits = [60, 100, 120]
3 capacity = 50
4
5 items = []
6
7 for i in range(len(weights)):
8     ratio = profits[i] / weights[i]
9     items.append((ratio, weights[i], profits[i]))
10
11 items.sort(reverse=True)
12
13 max_profit = 0
14
15 for ratio, weight, profit in items:
16     if capacity >= weight:
17         capacity -= weight
18         max_profit += profit
19     else:
20         max_profit += ratio * capacity
21         break
22
23 print("Maximum profit =", max_profit)
```



Maximum profit = 240.0

...Program finished with exit code 0
Press ENTER to exit console.

```
1 weights = [5, 10, 15]
2 profits = [30, 40, 45]
3 capacity = 20
4
5 items = []
6
7 for i in range(len(weights)):
8     ratio = profits[i] / weights[i]
9     items.append((ratio, weights[i], profits[i]))
10
11 items.sort(reverse=True)
12
13 max_profit = 0
14
15 for ratio, weight, profit in items:
16     if capacity >= weight:
17         capacity -= weight
18         max_profit += profit
19     else:
20         max_profit += ratio * capacity
21         break
22
23 print("Maximum profit =", max_profit)
```

Maximum profit = 85.0

...Program finished with exit code 0
Press ENTER to exit console.

main.py

```
1 jobs = ['P1', 'P2', 'P3', 'P4']
2 deadlines = [1, 1, 2, 2]
3 profits = [40, 30, 20, 10]
4
5 job_list = []
6
7 for i in range(len(jobs)):
8     job_list.append((profits[i], deadlines[i], jobs[i]))
9
10 job_list.sort(reverse=True)
11
12 max_deadline = max(deadlines)
13 slots = [None] * max_deadline
14
15 total_profit = 0
16
17 for profit, deadline, job in job_list:
18     for j in range(deadline - 1, -1, -1):
19         if slots[j] is None:
20             slots[j] = job
21             total_profit += profit
22             break
23
24 print("Selected jobs =", *[job for job in slots if job])
25 print("Maximum profit =", total_profit)
```

Selected jobs = P1 P3
Maximum profit = 60

...Program finished with exit code 0
Press ENTER to exit console.

main.py

```
1 jobs = ['A', 'B', 'C', 'D']
2 deadlines = [2, 1, 2, 1]
3 profits = [100, 19, 27, 25]
4
5 job_list = []
6
7 for i in range(len(jobs)):
8     job_list.append((profits[i], deadlines[i], jobs[i]))
9
10 job_list.sort(reverse=True)
11
12 max_deadline = max(deadlines)
13 slots = [None] * max_deadline
14
15 total_profit = 0
16
17 for profit, deadline, job in job_list:
18     for j in range(deadline - 1, -1, -1):
19         if slots[j] is None:
20             slots[j] = job
21             total_profit += profit
22             break
23
24 print("Selected jobs =", *[job for job in slots if job])
25 print("Maximum profit =", total_profit)
```

Selected jobs = C A
Maximum profit = 127

...Program finished with exit code 0
Press ENTER to exit console.


```
1 import heapq
2
3 characters = ['a', 'b', 'c', 'd', 'e', 'f']
4 frequencies = [5, 9, 12, 13, 16, 45]
5 heap = []
6 for i in range(len(characters)):
7     heapq.heappush(heap, (frequencies[i], characters[i]))
8 while len(heap) > 1:
9     freq1, char1 = heapq.heappop(heap)
10    freq2, char2 = heapq.heappop(heap)
11
12    new_node = (char1 + char2)
13
14    heapq.heappush(heap, (freq1 + freq2, new_node))
15
16 root = heap[0][1]
17
18 codes = {}
19
20 def generate_codes(node, code=""):
21     if len(node) == 1:
22         codes[node] = code
23         return
24
25     mid = len(node) // 2
26
27     generate_codes(node[:mid], code + "0")
28     generate_codes(node[mid:], code + "1")
29
30 generate_codes(root)
31 print("Huffman Codes:")
32 for char in characters:
33     print(char + ":", codes[char])
```

Huffman Codes:

a: 10
b: 110
c: 010
d: 011
e: 111
f: 00

...Program finished with exit code 0
Press ENTER to exit console.

```
1 jobs = ['A', 'B', 'C', 'D', 'E']
2 deadlines = [2, 1, 2, 1, 3]
3 profits = [100, 50, 10, 20, 30]
4
5 job_list = []
6
7 for i in range(len(jobs)):
8     job_list.append((profits[i], deadlines[i], jobs[i]))
9
10 job_list.sort(reverse=True)
11
12 max_deadline = max(deadlines)
13 slots = [None] * max_deadline
14
15 total_profit = 0
16
17 for profit, deadline, job in job_list:
18     for j in range(deadline - 1, -1, -1):
19         if slots[j] is None:
20             slots[j] = job
21             total_profit += profit
22             break
23
24 print("Selected jobs =", *[job for job in slots if job])
25 print("Maximum profit =", total_profit)
```

Selected jobs = B A E
Maximum profit = 180

...Program finished with exit code 0
Press ENTER to exit console.

main.py

```
1 import heapq
2 characters = ['x', 'y', 'z', 'w']
3 frequencies = [7, 8, 14, 25]
4 heap = []
5 for i in range(len(characters)):
6     heapq.heappush(heap, (frequencies[i], characters[i]))
7 while len(heap) > 1:
8     freq1, char1 = heapq.heappop(heap)
9     freq2, char2 = heapq.heappop(heap)
10
11     new_node = char1 + char2
12
13     heapq.heappush(heap, (freq1 + freq2, new_node))
14 root = heap[0][1]
15
16 codes = {}
17
18 def generate_codes(node, code=""):
19     if len(node) == 1:
20         codes[node] = code
21         return
22
23     mid = len(node) // 2
24
25     generate_codes(node[:mid], code + "0")
26     generate_codes(node[mid:], code + "1")
27
28 generate_codes(root)
29
30 print("Huffman Codes:")
31
32 for char in characters:
33     print(char + ":", codes[char])
```

▼ 🔍 📄 ⚙️ 📁

Huffman Codes:

x: 10
y: 11
z: 01
w: 00

...Program finished with exit code 0
Press ENTER to exit console. □


```
1 weights = [4, 8, 2, 6]
2 profits = [20, 40, 14, 30]
3 capacity = 10
4
5 items = []
6
7 for i in range(len(weights)):
8     ratio = profits[i] / weights[i]
9     items.append((ratio, weights[i], profits[i]))
10
11 items.sort(reverse=True)
12
13 max_profit = 0
14
15 for ratio, weight, profit in items:
16     if capacity >= weight:
17         capacity -= weight
18         max_profit += profit
19     else:
20         max_profit += ratio * capacity
21         break
22
23 print("Maximum profit =", max_profit)
```



Maximum profit = 54.0

...Program finished with exit code 0
Press ENTER to exit console.

```
1 weights = [2, 3, 5, 7]
2 profits = [10, 5, 15, 7]
3 capacity = 8
4
5 items = []
6
7 for i in range(len(weights)):
8     ratio = profits[i] / weights[i]
9     items.append((ratio, weights[i], profits[i]))
10
11 items.sort(reverse=True)
12
13 max_profit = 0
14
15 for ratio, weight, profit in items:
16     if capacity >= weight:
17         capacity -= weight
18         max_profit += profit
19     else:
20         max_profit += ratio * capacity
21         break
22
23 print("Maximum profit =", max_profit)
```



Maximum profit = 26.666666666666668

...Program finished with exit code 0
Press ENTER to exit console.

```
1 coins = [1, 2, 5, 10]
2 amount = 28
3
4 coins.sort(reverse=True)
5
6 result = []
7
8 for coin in coins:
9     while amount >= coin:
10         amount -= coin
11         result.append(coin)
12
13 print("Coins:", *result)
14 print("Minimum coins =", len(result))
```



```
Coins: 10 10 5 2 1
Minimum coins = 5
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

```
1 import heapq
2
3 characters = ['A', 'B', 'C', 'D']
4 frequencies = [10, 20, 30, 40]
5
6 heap = []
7
8 for i in range(len(characters)):
9     heapq.heappush(heap, (frequencies[i], characters[i]))
10
11 while len(heap) > 1:
12     freq1, char1 = heapq.heappop(heap)
13     freq2, char2 = heapq.heappop(heap)
14     new_node = char1 + char2
15     heapq.heappush(heap, (freq1 + freq2, new_node))
16 root = heap[0][1]
17 codes = {}
18 def generate_codes(node, code=""):
19     if len(node) == 1:
20         codes[node] = code
21         return
22
23     mid = len(node) // 2
24
25     generate_codes(node[:mid], code + "0")
26     generate_codes(node[mid:], code + "1")
27
28 generate_codes(root)
29
30 print("Huffman Codes:")
31
32 for char in characters:
33     print(char + ":", codes[char])
```



Huffman Codes:

A: 01

B: 10

C: 11

D: 00

...Program finished with exit code 0

Press ENTER to exit console.

```
1 coins = [1, 2, 5]
2 amount = 11
3
4 coins.sort(reverse=True)
5
6 result = []
7
8 for coin in coins:
9     while amount >= coin:
10         amount -= coin
11         result.append(coin)
12
13 print("Coins:", *result)
14 print("Minimum coins =", len(result))
```

Coins: 5 5 1
Minimum coins = 3

...Program finished with exit code 0
Press ENTER to exit console.


```

1 jobs = ['J1', 'J2', 'J3', 'J4', 'J5']
2 deadlines = [2, 1, 2, 1, 3]
3 profits = [20, 15, 10, 5, 1]
4
5 job_list = []
6
7 for i in range(len(jobs)):
8     job_list.append((profits[i], deadlines[i], jobs[i]))
9
10 job_list.sort(reverse=True)
11
12 max_deadline = max(deadlines)
13 slots = [None] * max_deadline
14
15 total_profit = 0
16
17 for profit, deadline, job in job_list:
18     for j in range(deadline - 1, -1, -1):
19         if slots[j] is None:
20             slots[j] = job
21             total_profit += profit
22             break
23
24 print("Selected jobs =", *[job for job in slots if job])
25 print("Maximum profit =", total_profit)

```



```

Selected jobs = J2 J1 J5
Maximum profit = 36

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

main.py

```
1 coins = [1, 3, 7, 10]
2 amount = 24
3
4 coins.sort(reverse=True)
5
6 result = []
7
8 for coin in coins:
9     while amount >= coin:
10         amount -= coin
11         result.append(coin)
12
13 print("Coins:", *result)
14 print("Minimum coins =", len(result))
```

Coins: 10 10 3 1
Minimum coins = 4

...Program finished with exit code 0
Press ENTER to exit console.